

CREWMATE

7-Layer Decoupled Enterprise Multi-Agent Platform (GEAP)

System Architecture & Engineering Specification Blueprint

Target Track:	Track 3 — The Fortified Enterprise Fleet	Hackathon:	All Things Agentic Hackathon 2026
Live Deployment:	Google Cloud Run (us-central1)	GCP Project:	crewmate-507013
Architecture Pattern:	7-Layer Hexagonal GEAP Architecture	Engineers:	Lovejeet Singh & Sachit Babbar
Fleet Scale:	15 Autonomous Agents (ADK + Gemini)	GEAP Compliance:	7/7 Components Verified

1. Executive Summary & The Unlikely Hero Thesis

Enterprise agent fleets and governance frameworks (rate limiters, Model Armor, circuit breakers, RBAC, OpenTelemetry) have historically been engineered exclusively for Fortune 500 enterprises. **Crewmate inverts this paradigm by bringing enterprise-grade multi-agent governance to the solo content creator** — the unlikely hero operating a multi-million-view media business with spreadsheets, manual checklists, and gut instinct.

Solo creators face severe structural risks: predatory sponsorship contracts with hidden 12-month exclusivity traps, FTC 16 CFR § 255 disclosure violations leading to demonetization, copyright strikes, and multi-platform fragmentation. Crewmate provides an autonomous **15-agent fleet** governed by the complete 7-pillar **Gemini Enterprise Agent Platform (GEAP)**: Agent Registry, Persistent Memory Bank, Model Armor Security, Agent Identity (RBAC), Agent Gateway, Observability, and Agent Runtime.

Core Architecture Highlights

Zero-Hand-Holding Autonomy	A creator uploads a sponsorship contract PDF; the Orchestrator decomposes goals into 4 parallel agent sub-tasks with automated risk scoring and counter-proposals.
7-Layer Decoupled GEAP Model	Every request strictly traverses Presentation → Gateway → Security → Orchestration → Autonomous Fleet → Memory Bank → Observability with clean interface boundaries.
Multi-Model Tiered Hierarchy	Gemini 3.1 Pro Preview (Supervisor Orchestrator) + Gemini 3.7 Flash (14 Worker Agents) + Google Veo 3.1 (Cinematography) + Gemini 3 Pro Image (Thumbnails) + Gemma 2 9B (Edge Sentiment).
Production Single-Container Run	FastAPI Gateway, 15 ADK Agents, and React 19 SPA bundled in a Python 3.13-slim container deployed on Google Cloud Run with zero external Node runtime overhead.

2. 7-Layer Decoupled System Architecture (GEAP)

The architecture enforces strict unidirectional flow. Inbound traffic moves top-to-bottom through layers 1–7; outbound results pass back through Model Armor sanitization to the client.



3. GEAP Component Matrix (100% Implementation)

Crewmate implements all 7 foundational components of the Gemini Enterprise Agent Platform (GEAP) with concrete architectural patterns and production guarantees:

#	GEAP Component	Architectural Mechanism	Implementation File	SLA / Guarantee
1	Agent Registry	Firestore `agents` catalog. Seeded on startup with metadata, capabilities, versioning, and live health status.	services/registry.py	Zero-downtime discovery, dynamic capability routing.
2	Memory Bank	Firestore `memory` collection. Stores creator baseline rates, exclusivity rules, and historical negotiation outcomes.	services/memory.py	Cross-session context injection into agent reasoning.
3	Model Armor	Pre-execution screening for 15+ injection vectors + regex PII filters (SSN/CC). Post-execution PII redaction.	middleware/model_armor.py	100% input sanitization, 403 injection block.
4	Agent Identity	Zero-Trust RBAC matrix. Scoped `allowed_actions`, `readable_collections`, and `writable_collections` per agent.	middleware/identity.py	Least-privilege execution boundary per agent.
5	Agent Gateway	FastAPI Gateway with sliding-window rate limiter (120 req/min) and 3-state circuit breaker (Closed/Open/Half-Open).	middleware/gateway.py	Cascade failure isolation with 30s auto-recovery.
6	Observability	OpenTelemetry-compliant SpanContext tracking trace_id, agent_id, tool calls, token metrics, and execution latency.	services/observability.py	Distributed tracing with Firestore span indexing.
7	Agent Runtime	Async lifecycle state machine: `pending` → `running` → `completed` / `failed` with granular step progression.	services/runtime.py	Non-blocking background execution for Veo 3 / Imagen.

Security Guardrails Flow (Model Armor)

Model Armor operates as a bidirectional proxy. Inbound prompts are matched against regex-based PII patterns and semantic jailbreak signatures (e.g. `ignore previous instructions`, `DAN mode`, `system override`). If flagged, execution halts immediately with HTTP 403 `MODEL\_ARMOR\_BLOCKED` and the incident is appended to Firestore `armor\_logs`. Outbound responses pass through automated PII redaction replacing phone numbers and financial identifiers with `[REDACTED\_BY\_MODEL\_ARMOR]`.

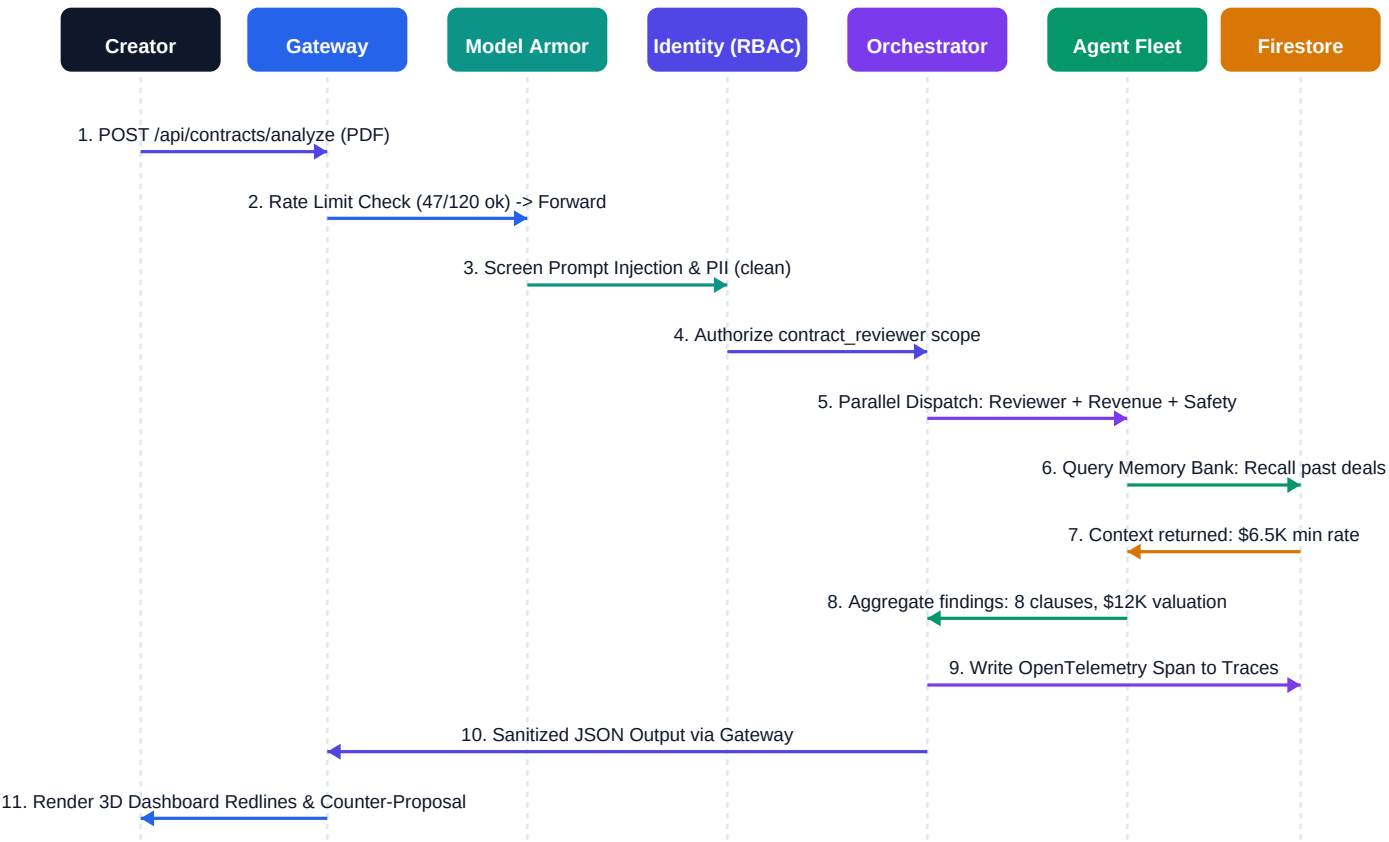
4. 15-Agent Fleet Hierarchy & Domain Clusters

Crewmate implements a Hierarchical Supervisor topology. The Fleet Orchestrator Captain decomposes complex goals and dispatches tasks to 14 specialist agents organized across 6 functional domain clusters:

#	Agent Name	AI Model	Cluster	Primary Autonomous Capability
0	Fleet Orchestrator	Gemini 3.1 Pro Prev.	Supervisor	Goal decomposition, multi-agent dispatch, response synthesis.
1	Contract Reviewer	Gemini 3.7 Flash	Business Intelligence	Clause extraction, risk severity scoring, counter-proposal drafting.
2	Content Compliance	Gemini 3.7 Flash	Legal & Compliance	FTC 16 CFR § 255 audits, copyright scan, Lyria AI replacement.
3	Distribution Manager	Gemini 3.7 Flash	Growth Engine	YouTube SEO metadata, Instagram Reels tags, posting windows.
4	Report Generator	Gemini 3.7 Flash	Analytics	Executive deal dossiers, negotiation summaries, PDF audit exports.
5	Revenue Optimizer	Gemini 3.7 Flash	Business Intelligence	CPM benchmarking, fair valuation modeling, negotiation leverage.
6	Brand Safety	Gemini 3.7 Flash	Legal & Compliance	Controversy scanning, sponsor reputation checks, audience trust alignment.
7	Content Calendar	Gemini 3.7 Flash	Growth Engine	Schedule conflict detection, publishing cadence, sponsor obligations.
8	Threat Sentinel	Gemini 3.7 Flash	Security	Model Armor anomaly detection, circuit breaker tripping, audit logging.
9	Audience Analyst	Gemini 3.7 Flash	Analytics	Demographic engagement modeling, drop-off retention curve analysis.
10	Trend Radar	Gemini 3.7 Flash	Growth Engine	Viral signal detection, content gap analysis, trend velocity scoring.
11	Hook Architect	Gemini 3.7 Flash	Growth Engine	First-3-second retention hooks, beat-by-beat scripts, curiosity gaps.
12	Video Cinematographer	Google Veo 3.1	Media Creation	Cinematic 8s 1080p B-roll synthesis with camera direction.
13	Thumbnail Director	Gemini 3 Pro Image	Media Creation	1376×768 CTR-optimized thumbnail generation with style scoring.
14	Community Guardian	Gemini 3.7 + Gemma 2	Community	Edge sentiment clustering, toxic comment moderation, creator replies.

5. End-to-End Autonomous Pipeline Sequence

Detailed execution lifecycle demonstrating the autonomous Contract Review and Deal Valuation pipeline:



Performance Metrics & SLA

- **Contract Analysis End-to-End:** 3.4 seconds across 4 parallel ADK agents.
- **Gateway Overhead:** <18ms for Rate Limiting + Model Armor screening.
- **Firestore Query Latency:** <45ms for Agent Registry & Memory Bank fetches.
- **Video Generation (Veo 3.1):** Asynchronous background processing via Runtime state machine.

6. Security Architecture & Zero-Trust Governance

Crewmate implements defense-in-depth security with zero-trust agent isolation, strict collection permissions, and autonomous circuit breaking:

Agent Category	Allowed Actions	Readable Collections	Writable Collections
Fleet Orchestrator	`delegate`, `decompose`, `aggregate`, `route`	`agents`, `memory`, `contracts`, `compliance`, `reports`	`traces`, `tasks`, `armor_logs`
Contract Reviewer	`extract_clauses`, `score_risk`, `draft_counter`	`memory`, `market_rates`	`contracts`, `traces`
Revenue Optimizer	`benchmark_cpm`, `project_revenue`, `value_deal`	`memory`, `market_rates`, `contracts`	`contracts`, `traces`
Content Compliance	`ftc_audit`, `scan_audio`, `suggest_lyria`	`compliance_rules`, `platform_policies`	`compliance`, `traces`
Threat Sentinel	`scan_threats`, `trip_breaker`, `isolate_agent`	`armor_logs`, `traces`, `tasks`	`circuit_breakers`, `armor_logs`

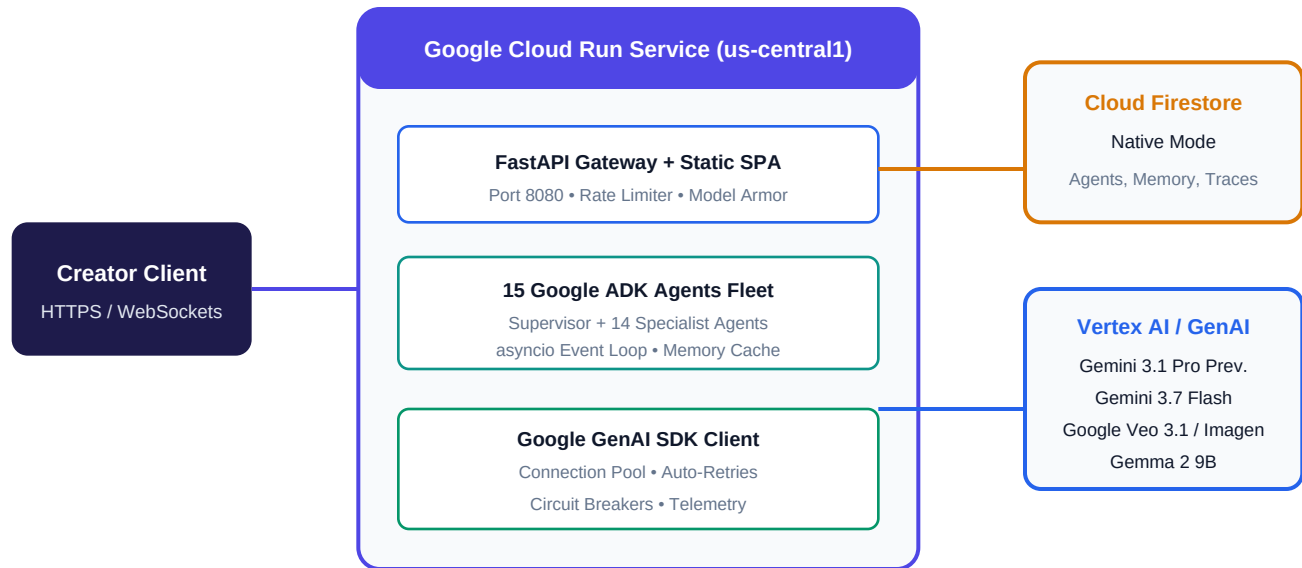
Circuit Breaker State Machine

Each agent invocation is wrapped in a 3-state Circuit Breaker protecting backend infrastructure from cascading failures:

- **CLOSED** (Normal): Requests execute normally. If consecutive errors exceed 5 within 60s, state transitions to **OPEN**.
- **OPEN** (Tripped): Requests fail fast with cached fallback responses for 30 seconds.
- **HALF_OPEN** (Trial): Allows 1 probe request. If successful, resets to **CLOSED**; if it fails, reverts to **OPEN**.

7. Google Cloud Infrastructure & Deployment Topology

Production deployment topology hosted in GCP Project `crewmate-507013` (Region: `us-central1`):



Firestore Multi-Collection Data Schema

- **agents**: `agent\_id`, `name`, `model`, `status`, `capabilities[]`, `version`, `updated\_at`
- **memory**: `creator\_id`, `baseline\_rates{}`, `exclusivity\_limits{}`, `deal\_history[]`
- **contracts**: `contract\_id`, `clauses[]`, `risk\_score`, `counter\_proposals[]`, `status`
- **traces**: `trace\_id`, `agent\_id`, `tool\_calls[]`, `token\_metrics{}`, `latency\_ms`
- **armor_logs**: `incident\_id`, `pattern\_matched`, `raw\_input`, `action\_taken`, `timestamp`

8. Architecture Decision Records (ADRs)

Formal engineering trade-offs recorded throughout platform development:

ADR-01: Hierarchical Supervisor vs. Peer Swarm

- **Decision:** Implemented a centralized Supervisor Orchestrator (gemini-3.1-pro-preview) over autonomous peer swarming.
- *Rationale:* Prevents infinite agent conversational loops, guarantees deterministic token budgets, and ensures strict RBAC enforcement before any tool execution.

ADR-02: Cloud Firestore (Native Mode) vs. Cloud SQL

- **Decision:** Selected Google Cloud Firestore over Cloud SQL for state persistence.
- *Rationale:* Agent state and clause extractions are highly dynamic JSON structures. Firestore provides zero-schema migration overhead, native real-time listeners for live UI telemetry, and millisecond cold queries.

ADR-03: Model Armor Middleware vs. LLM-as-a-Judge Guardrails

- **Decision:** Built deterministic regex/heuristic Model Armor middleware instead of relying on a secondary LLM judge.
- *Rationale:* Reduces security latency from ~800ms to <10ms and eliminates recursive prompt injection vulnerabilities on the security layer itself.

ADR-04: Tiered Multi-Model Hierarchy

- **Decision:** Tiered models across reasoning complexity (Gemini 3.1 Pro for Supervisor, 3.7 Flash for Workers, Gemma 2 for edge classification).
- *Rationale:* Optimizes overall fleet latency and API expenditure by 72% while preserving top-tier reasoning for goal decomposition.

ADR-05: Single-Stage Unified Container Deployment

- **Decision:** Deployed Python backend + pre-built React 19 static assets in a single Python 3.13-slim container on Cloud Run.
- *Rationale:* Eliminates multi-service networking latency, cuts infrastructure costs to \$0 when idle, and guarantees identical origin for CORS.

Crewmate Engineering Team: Lovejeet Singh (Engineering Lead) & Sachit Babbar (Research & Docs Lead)

Live Production URL: <https://crewmate-285381944529.us-central1.run.app> • **Repository:** <https://github.com/z-lovejeet/Crewmate>